

Generative Causality-Driven Network for Graph Multi-Task Learning

Xixun Lin , Qing Yu , Yanan Cao , Lixin Zou, Chuan Zhou , *Member, IEEE*, Jia Wu , *Senior Member, IEEE*, Chenliang Li , Peng Zhang , and Shirui Pan , *Senior Member, IEEE*

Abstract—Multi-task learning (MTL) is a standard learning paradigm in machine learning. The central idea of MTL is to capture the shared knowledge among multiple tasks for mitigating the problem of data sparsity where the annotated samples for each task are quite limited. Recent studies indicate that graph multi-task learning (GMTL) yields the promising improvement over previous MTL methods. GMTL represents tasks on a task relation graph, and further leverages graph neural networks (GNNs) to learn complex task relationships. Although GMTL achieves the better performance, the construction of task relation graph heavily depends on simple heuristic tricks, which results in the existence of spurious task correlations and the absence of true edges between tasks with strong connections. This problem largely limits the effectiveness of GMTL. To this end, we propose the Generative Causality-driven Network (GCNet), a novel framework that progressively learns the causal structure between tasks to discover which tasks are beneficial to be jointly trained for improving generalization ability and model robustness. To be specific, in the feature space, GCNet first introduces a feature-level generator to generate the structure prior for reducing learning difficulty. Afterwards, GCNet develops a output-level generator which is parameterized as a new causal energy-based model (EBM) to refine the learned structure prior in the output space driven by causality. Benefiting from our proposed causal framework, we theoretically derive an intervention contrastive estimation for training this causal EBM efficiently. Experiments are conducted on multiple synthetic and real-world

datasets. Extensive empirical results and model analyses demonstrate the superior performance of GCNet over several competitive MTL baselines.

Index Terms—Graph machine learning, graph multi-task learning, graph neural networks.

I. INTRODUCTION

WITH the rise of powerful computing resources, large-scale training data and representative neural networks, deep learning (DL) [1] has achieved remarkable results in many applications, such as intelligent recommendation [2], information retrieval [3] and autonomous driving [4]. Albeit their enormous successes, DL-based approaches usually require a huge amount of annotated data for model training which is far from sufficient for some research fields, such as chemical synthesis [5] and drug discovery [6], where obtaining expert annotation is exceedingly expensive. Multi-task learning (MTL) [7], [8], [9] offers an effective solution to alleviate this data sparsity problem by capturing the supervision or knowledge that can be shared by multiple tasks. Although the number of annotated data of each task may be insufficient, MTL can exploit the labeled data of all tasks from the view of data augmentation to obtain a more accurate learner of each task [10], [11], [12].

A critical challenge of MTL is to learn the relation among tasks for determining how to share the learned knowledge. In recent years, many MTL approaches [13], [14], [15], [16], [17] are devoted to attain this goal. Graph multi-task learning (GMTL) [18], [19], [20] is one of promising directions, where the relation between tasks has already been given in the form of a task relation graph via some pre-defined heuristic rules. Previous GMTL works have focused on utilizing the graph information as the inductive bias to regularize the learning process of multiple tasks. Recently, SGNN-EBM [20] exhibits the better performance than previous methods. Instead of predicting each task independently, SGNN-EBM formulates MTL as a structured prediction problem [21] and learns the joint distribution of task labels with the given task relation graph.

However, in fact, deriving the true relation between tasks requires substantial manual evaluations. The number of relations grows polynomially as the number of tasks increases, making it unaffordable in practice. As a workaround, most previous methods [20], [22], [23], [24] construct the task relation graph via some straightforward clustering strategies or statistical correlations between tasks from relevant domains. Obviously, this

Received 12 February 2025; revised 25 August 2025; accepted 9 September 2025. Date of publication 16 September 2025; date of current version 3 December 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB3102200, in part by the National Natural Science Foundation of China under Grant 62402491, Grant 62376064, Grant U23A20305, and Grant 62472416, and in part by the China Postdoctoral Science Foundation under Grant 2025M771524. Recommended for acceptance by P. Zhao. (*Corresponding authors: Yanan Cao; Peng Zhang.*)

Xixun Lin and Yanan Cao are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: linxixun@iie.ac.cn; caoyanan@iie.ac.cn).

Qing Yu, Lixin Zou, and Chenliang Li are with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430079, China (e-mail: yu_qing@whu.edu.cn; zoulixin@whu.edu.cn; cllee@whu.edu.cn).

Chuan Zhou is with the Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: zhouchuan@amss.ac.cn).

Jia Wu is with the Department of Computing, Macquarie University, Sydney, NSW 2113, Australia (e-mail: jia.wu@mq.edu.au).

Peng Zhang is with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510006, China (e-mail: p.zhang@gzhu.edu.cn).

Shirui Pan is with the School of Information and Communication Technology, Griffith University, Gold Coast, QLD 4215, Australia (e-mail: s.pan@griffith.edu.au).

Digital Object Identifier 10.1109/TPAMI.2025.3610096

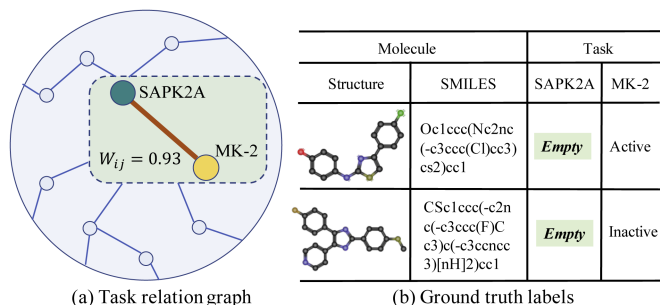


Fig. 1. A simple example of MTL for molecular property prediction. The high score W_{ij} between ‘SPAK2A’ and ‘MK-2’ is inferred by some heuristic rules from protein domains [25]. However, in (b), the state ‘Empty’ from the ground truth indicates that the biological property ‘SPAK2A’ is missing or unpredictable, indicating the edge between these two tasks shown in (a) is unfaithful.

graph construction method will incur spurious edges between unrelated tasks and overlook true edges that should exist between tasks with strong connections. An empirical example of molecular property prediction¹ is provided in Fig. 1. In Fig. 1(a), each node denotes a task that measures a certain biological effect of molecules. Although the task relation graph demonstrates that ‘SPAK2A’ has a strong connection with ‘MK-2’ due to unknown confounding factors, these two tasks are actually unrelated to each other w.r.t. the given molecules. Both the presence of spurious edges and the absence of true edges can produce negative effects on MTL. Particularly, *spurious edges will allow information to be shared between two unrelated tasks, which can worsen the performance of both; and the absence of true edges will make the two related tasks unable to train together to advance one another.*

To address these issues, we propose the Generative Causality-driven Network (GCNet), a novel framework for recognizing the causal edges from the task relation graph and maximizing the causal effect between tasks on learning the joint distribution of task labels. To be specific, GCNet follows a progressive learning process that owns two phases. In the first phase, GCNet contains a feature-level generator to generate the structure prior via aligning task labels in the feature space. This is an essential learning step, because directly learning the causal structure in the high-dimensional task space is very hard or even intractable [26]. In the second phase, a output-level generator, which is parameterized a causal energy-based model (EBM), is introduced to achieve the structured prediction. The causal EBM includes a structural equation model (SEM) layer to further refine the learned structure prior, with the goal of capturing the dependency of all task predictions in the output space driven by causality. Furthermore, since GCNet can perform the ‘do-operation’ to generate counterfactual energy scores, which naturally tunes the noise data distribution. We can leverage this characteristic to design a novel intervention contrastive estimation for optimizing the causal EBM efficiently. In general, the main contributions of our work are summarized here:

¹Since the labeled data in drug discovery is very limited and is expensive to obtain, the problem of molecular property prediction provides an ideal test environment for MTL. A detailed description of this part is provided in Section A.

- We have innovatively developed a generative framework termed GCNet, which can progressively perform the causal structure learning on MTL from the feature and output spaces. The goal is to identify which tasks are advantageous for joint training, thereby enhancing generalization ability and model robustness.
- We introduce a causal EBM that can reveal the causal structure among tasks and supports the intervention operations of energy scores for highly effective model training in a manner of contrastive learning.
- We conduct extensive experiments on six datasets including one synthetic dataset and five real-world datasets under different data scenarios. Extensive empirical results and detailed analyses validate the superiority of GCNet over its strong counterparts.

II. RELATED WORK

A. Multi-Task Learning

MTL provides a principal way to alleviate the data sparsity problem by learning the generalized knowledge that can be shared by all tasks [12]. A significant problem that has always been discussed is how to capture the relation between different tasks. MGP [27] uses the Gaussian process to model inter-task correlations with a shared covariance function, which enables model to learn the task relation from data automatically. MTRL [28] considers a regularization way to describe positive and negative task correlations. The follow-up work MRN [15] applies a tensor-variate normal distribution as a prior on model parameters for discovering task relations. MMoe [29] introduces the Mixture-of-Experts (MoE) structure to MTL where expert sub-models are shared by all tasks and each gating network is optimized by the task-specific information.

A popular research branch of MTL is *graph multi-task learning (GMTL)* which aims to model the task relation effectively when the relation between tasks can be organized as a graph by some external resources. Early GMTL works [18], [23], [30] exploit the given task relation to regularize the learning process of multiple tasks. For instance, Kato et al [31] apply the graph regularization to support vector machines (SVMs) and cast the optimization problem as a second order cone program which can be solved efficiently. However, most of them are shallow models and the model expressiveness is limited for learning the complicated real-world data. Recently, SGNN-EBM [20] leverages a state graph neural network (SGNN) with the energy-based model (EBM) to model the task relation graph, and this deep model achieves state-of-the-art (SOTA) performance compared with previous baselines. But the constructed task relation graph is not reliable, heavily relying on such a flawed graph would lead to sub-optimal results for current GMTL approaches.

B. Graph Neural Networks

GNNs have become a standard learning paradigm for handling graph-structured data [32], [33], [34], [35]. Popular GNN architectures include spectral-based and spatial-based methods, such as SpectralCNN [36] ChebyNet [37], graph convolutional

network (GCN) [38], GAT [39] and GraphSAGE [40]. Spectral-based GNNs transform node features into weighted sums of different eigenvectors via graph Fourier transform, which is good at capturing global information, while spatial-based GNNs update node features via the neighbor aggregation mechanism, which have lower computational complexity and better model flexibility. These GNNs have achieved impressive success in many real-world applications, e.g., computational chemistry [41], [42], recommender systems [43], [44], social networks [45], [46] and so on.

Here we list several representative GNNs that have a close relationship with our method. To be specific, MPNN [47] introduces a general GNN framework that owns message, update, and output phases for aggregating node and edge features and making prediction. GIN [48] investigates the connection between GNNs and the Weisfeiler-Lehman (WL) graph isomorphism test [49], and uses an injective summation operation to aggregate neighbor node features for improving the expressiveness of GNNs. GMNN [50] combines GNNs with conditional random fields (CRFs) [51] to achieve structured prediction for modeling the structured output of node labels. GMNN introduces an effective variational expectation-maximization (EM) framework [52], [53], [54] to optimize model parameters. SPN [55] further extends GMNN to finish structured prediction. SPN includes two types of GNNs to model the pseudo-marginal label distributions of nodes and edges respectively for constructing a near-optimal joint label distribution which is much easier to be optimized.

C. Causal Discovery

Discovering the causal structure from observed data has been widely studied over the past few years and demonstrates a significant impact on a series of applications [56], [57], [58], [59]. The goal of causal discovery is to learn a directed acyclic graph (DAG) between variables that can provide better explanations of out-of-distribution data. For the learned DAG, the structural equation model (SEM) describes the functions of the learned causal edges [60]. A traditional way to discover causal relations is to use interventions or randomized experiments, but the main challenge is the unaffordable cost of searching over the non-convex combinatorial search space [26].

To address this challenge, NOTEARS [61] reformulates this combinatorial optimization problem as a continuous optimization program with fully differentiable DAG constraints, and they demonstrate the effectiveness of the score-based objective in the form of linear SEM with a least-squares loss. Zhu et al [62] consider a reinforcement learning-based method to search for this DAG with the best score according to a pre-defined score function. CASTLE [63] introduces a regularization-based method to learn the causal graph between neural network parameters for improving the generalization performance on out-of-sample predictions. Instead of adopting the maximum-likelihood point estimation of DAGs from observed data, BCD Nets [64] proposes a variational inference-based method to estimate the distribution over DAGs. Different from above methods, we are the first work that focuses on learning the causal structure among tasks

TABLE I
SUMMARY OF USED NOTATIONS

Notation	Definition or Description
$\mathbf{x}, \mathbf{y}$	the input data and the task label vector
m, o	the numbers of tasks and of classes
G	the constructed task relation graph
$\mathbf{A}$	the adjacency matrix of G
$\mathbf{C}$	the generated confidence matrix of $\mathbf{A}$
$\mathbf{H}^0$	the initial task attributes
G^c	the causal structure of G
$\mathbf{A}^c$	the adjacency matrix of G^c
$k(\mathbf{x})$	the input embedding of $\mathbf{x}$
$\mathbf{Z}_i$	the task embedding of the task i
$\mathbf{Q}_i^l(\mathbf{x})[s]$	the node-level state embedding of the task i in the l -th layer
$\mathbf{U}_{ij}^l(\mathbf{x})[s, s']$	the edge-level state embedding of the task pair (i, j) in the l -th layer
$f_i(\mathbf{x})[y_i]$	the first-order scalar factor of the task i
$f_{ij}(\mathbf{x})[y_i, y_j]$	the second-order scalar factor of the task pair (i, j)

from the feature and output spaces progressively. GOLEM [65] explores sparsity and DAG constraints in learning DAG models through unconstrained optimization. CO-NOD [66] develops a nonparametric approach to find the causal skeleton and directions, and estimates the properties of mechanism changes from heterogeneous data.

III. PRELIMINARY

A. Problem Formulation

We start from the basic notations of MTL in the scenario of *supervised learning* where most MTL studies fall in this setting [12]. The used notations have been summarized in Table I. We first introduce the definitions of MTL and GMTL, and then give a real-world example of GMTL.

Definition 1 (Multi-task learning): Given m learning tasks and the input data $\mathbf{x}$, the goal of MTL is to model the joint label distribution $p(\mathbf{y}|\mathbf{x})$ where $\mathbf{y} := (y_1, y_2, \dots, y_m)$ with y_i being the label of the i -th task that aims to complete the i -th property prediction. Each y_i owns o independent classes, i.e., $y_i \in \{0, 1, \dots, o-1\}$, $\forall i \in \{1, 2, \dots, m\}$. By learning m tasks together, MTL can use the shared knowledge among tasks to improve the prediction of every single task.

Definition 2 (Graph multi-task learning): Building upon MTL, GMTL introduces a constructed task relation graph $G = (V, E)$ where V is the node set of all tasks ($|V| = m$) and E is the edge set that represents the constructed relations between tasks. G can be also depicted by the adjacency matrix $\mathbf{A} \in \mathbb{R}^{m \times m}$ where $A_{ij} = 1$ if the task pair $(i, j) \in E$ otherwise $A_{ij} = 0$ ($\mathbf{A}$ is not limited to a binary form, and it can be also defined as a weighted matrix). The joint distribution of task labels is now $p(\mathbf{y}|\mathbf{x}, G)$ which is conditioned on G .

Here we provide a real-world GMTL problem, i.e., molecular property prediction [20], [67]. $\mathbf{x}$ represents a molecule and each task refers to a binary classification ($o = 2$) that measures a certain biological effect, e.g., inhibition or activation of proteins. The task relation graph G can be roughly constructed by using the protein correlation [25] and heuristic rules [20], resulting in an unfaithful structure among tasks that needs to be updated.

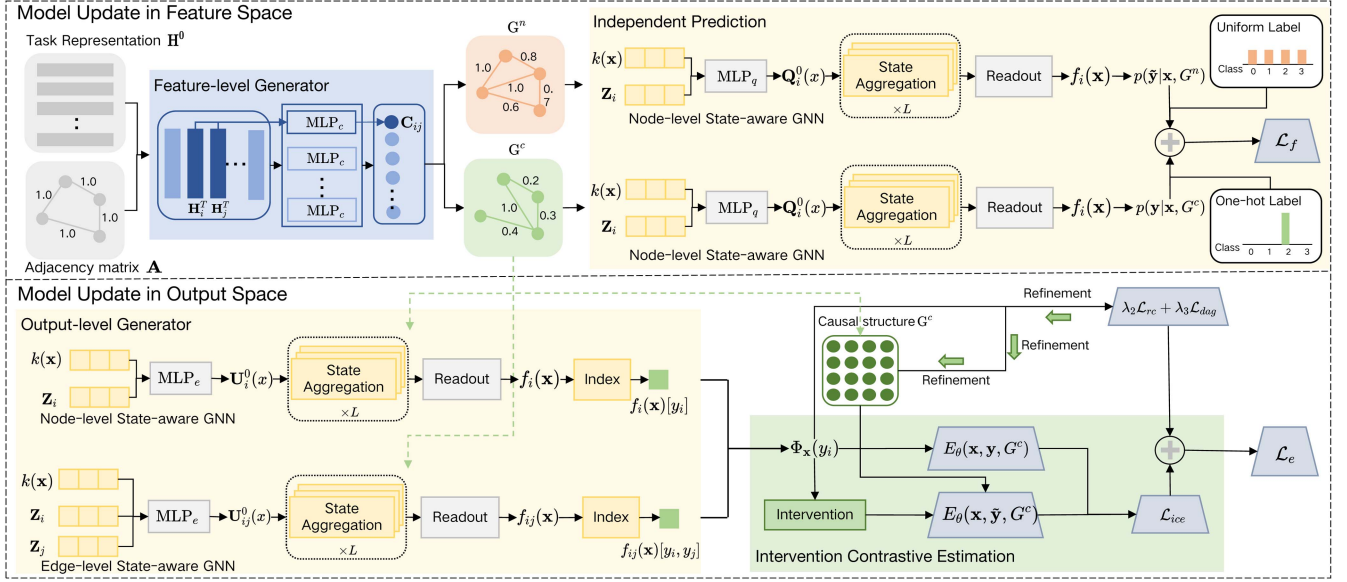


Fig. 2. The training framework of GCNet. In the feature space, we first leverage the feature-level generator to distinguish the informative and non-informative structure priors from the constructed task relation graph; then the output-level generator, which is parameterized as the causal EBM, refines the informative structure prior to generate the useful structure among tasks in the output space driven by causality. GCNet follows a two-phase training mode to optimize two types of generators progressively. The model update of feature-level generator in the feature space is guided by the independent prediction, and the model update of output-level generator is optimized via the proposed intervention contrastive estimation.

In GCNet, we attempt to recognize the causal structure G^c from G to facilitate the generalization ability and model robustness of current GMTL approaches. The target function of our model is therefore to learn the joint label distribution $p(\mathbf{y}|\mathbf{x}, G^c)$.

B. Energy-Based Models

Energy-based models (EBMs) [68] are a family of probability generative models, which specify the probability density via a parameterized energy function $E_\theta(\mathbf{x}, \mathbf{y})$ mapping each input $(\mathbf{x}, \mathbf{y})$ to a single scalar known as the energy [68]. EBMs are very attractive in functional form because $E_\theta(\mathbf{x}, \mathbf{y})$ merely represents an unnormalized negative log-probability that can be defined by any expressive deep neural networks (DNNs). A set of energy scores can be used to derive the probability $p_\theta(\mathbf{y}|\mathbf{x})$ via the Gibbs distribution:

$$p_\theta(\mathbf{y}|\mathbf{x}) = \frac{e^{-E_\theta(\mathbf{x}, \mathbf{y})}}{\Gamma_\theta(\mathbf{x})} = \frac{e^{-E_\theta(\mathbf{x}, \mathbf{y})}}{\sum_{\mathbf{y}' \in \mathbb{Y}} e^{-E_\theta(\mathbf{x}, \mathbf{y}')}}, \quad (1)$$

where $\Gamma_\theta(\mathbf{x}) := \sum_{\mathbf{y}' \in \mathbb{Y}} e^{-E_\theta(\mathbf{x}, \mathbf{y}')} d\mathbf{y}'$ is the partition function which marginalizes over $\mathbf{y}$ in the label space $\mathbb{Y} = \{0, 1, \dots, o-1\}^m$. Current works [20], [21] draw a connection of GMTL and EBMs, aiming to exploit the flexible expression of $E_\theta(\mathbf{x}, \mathbf{y})$ to further model the task relations in the output space. However, maximizing $p_\theta(\mathbf{y}|\mathbf{x})$ involves an intractable integral in $\Gamma_\theta(\mathbf{x})$, making the training EBMs particularly difficult. In Section B2, we theoretically derive a new intervention contrastive estimation to solve this critical issue.

IV. METHODOLOGY

In this section, we provide a detailed description of the generative causality-driven network (GCNet) which follows a

progressive learning process with a two-phase model update. In the first phase, GCNet uses a feature-level generator to induce the structure prior from the constructed task relation graph in the feature space. In the second phase, GCNet introduces an output-level generator parameterized as a causal EBM to recover the causal effects of energy factors in the energy function, which can generate the causal structure in the output space based on the learned structure prior. The overall training framework of GCNet is shown in Fig. 2.

A. Phase 1: Model Update in Feature Space

1) *Feature-Level Generator*: To reduce the difficulty of learning the causal structure in the high-dimensional task space, we first learn the structure prior from the constructed task relation graph. Afterwards, the learned structure prior is refined by the causal EBM to generate the final causal structure in the output space. Concretely, we propose a feature-level generator to recognize the informative and non-informative structure priors, i.e., $G^c = \{V, E^c\}$ and $G^n = \{V, E^n\}$.² In particular, we first use a two-layer graph convolutional network (GCN) [38] to encode the task relation graph G :

$$\mathbf{H} = \hat{\mathbf{A}} \text{ReLU}(\mathbf{A} \mathbf{H}^0 \mathbf{W}^0) \mathbf{W}^1, \quad (2)$$

where $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$, $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$, $\mathbf{D}$ is the degree matrix, $\mathbf{H}^0$ represents initial task attributes, $\{\mathbf{W}^0, \mathbf{W}^1\}$ denotes the model parameters, and $\mathbf{H} \in \mathbb{R}^{m \times d_1}$ is the task embedding matrix. Afterwards, our generator produces a confidence matrix $\mathbf{C} \in \mathbb{R}^{m \times m}$ that assigns each possible task pair a value indicating its estimated impact on the generation of

²We also denote the informative structure prior as G^c to simplify notation.

the informative structure prior. Given a task pair (i, j) , we define the element C_{ij} as follows,

$$C_{ij} = \tanh(\text{MLP}_c(\mathbf{H}_i^T + \mathbf{H}_j^T)), \quad (3)$$

where T denotes the transpose operation, $\mathbf{H}_i^T$ and $\mathbf{H}_j^T$ are the column feature vectors of task i and task j , respectively. $\text{MLP}_c(\cdot)$ denotes a multi-layer perceptron (MLP) function. Finally, the informative structure prior $\mathbf{A}^c$ is defined as:

$$\mathbf{A}^c = \mathbf{A} + \mathbf{C}, \text{ s.t. } 0 \leq \mathbf{A}_{ij}^c \leq 1, \forall 1 \leq i, j \leq m. \quad (4)$$

The min-max normalization is applied to (4) for satisfying the above constraint. The non-informative structure prior G^n is mutually exclusive with G^c , which can be given as $\mathbf{A}^n = \mathbf{J} - \mathbf{A}^c$ where $\mathbf{J}$ denotes an all-one matrix.

2) *Independent Prediction*: Based on G^c , the joint label distribution is decomposed into the following equation with the independent assumption:

$$p_\theta(\mathbf{y}|\mathbf{x}, G^c) = \prod_{i=1}^m p_\theta(y_i|\mathbf{x}, G^c). \quad (5)$$

$p_\theta(y_i|\mathbf{x}, G^c)$ represents the label distribution of the i -th task. To finish the prediction of all tasks, we first embed the input data and all tasks into the feature space. Depending on the specific data form of $\mathbf{x}$, we can utilize different encoders to learn the input embedding $k(\mathbf{x})$, and the specific encoder implementation is provided in Section 2.

As for tasks, we use another two-layer GCN to encode the learned G^c , and the generated node embeddings are taken as task embeddings $\{\mathbf{Z}_i\}_{i=1}^m$. Inspired by SGNN [20], we then propose a node-level state-aware GNN to model the high-level interaction between $k(\mathbf{x})$ and $\{\mathbf{Z}_i\}_{i=1}^m$ on G^c . The input feature of each node is constructed as:

$$\mathbf{Q}_i^0(\mathbf{x}) = \text{MLP}_q^0([k(\mathbf{x})|\mathbf{Z}_i]). \quad (6)$$

Here $\mathbf{Q}_i^0(\mathbf{x}) = \{\mathbf{Q}_i^0(\mathbf{x})[s]\}_{s=0}^{o-1}$ is disentangled into each state representation $\mathbf{Q}_i^0(\mathbf{x})[s] \in \mathbb{R}^{d_2}$, and each state s corresponds to a specific class. In the message-passing propagation, only nodes with the same state will be interacted. The calculation in the l -th layer is given as:

$$\mathbf{Q}_i^{l+1}(\mathbf{x})[s] = \text{MLP}_q^{l+1}\left(\mathbf{Q}_i^l(\mathbf{x})[s] + \sum_{j \in N(i)} \mathbf{A}_{ij}^c \mathbf{Q}_j^l(\mathbf{x})[s]\right), \quad (7)$$

where $N(i)$ denotes the one-hop neighborhood of the task i in $\mathbf{A}^c$. After L propagations, we aggregate all states into a first-order factor for making prediction of the task i :

$$f_i(\mathbf{x}) = R([|\mathbf{Q}_i^L(\mathbf{x})[0]|, \dots, |\mathbf{Q}_i^L(\mathbf{x})[o-1]|], l=1, \dots, L) \quad (8)$$

where $R(\cdot)$ is a readout function implemented by MLP layers. The label distribution is finally defined as $p_\theta(y_i|\mathbf{x}, G^c) = \text{softmax}(f_i(\mathbf{x}))$.

The non-informative structure prior G^n is involved in estimating a pseudo-label distribution of each task which can be defined

as a uniform distribution over o classes, i.e., $p(\tilde{y}_i) \sim \text{unif}\{0, o-1\}$. The joint pseudo-label distribution is given as:

$$p_\theta(\tilde{\mathbf{y}}|\mathbf{x}, G^n) = \prod_{i=1}^m p_\theta(\tilde{y}_i|\mathbf{x}, G^n). \quad (9)$$

The objective function is to maximize the following log-likelihood:

$$\mathcal{L}_f = \sum_{i=1}^m p_\theta(y_i|\mathbf{x}, G^c) + p_\theta(\tilde{\mathbf{y}}|\mathbf{x}, G^n) - \lambda_1 \frac{\|\mathbf{A}^c\|_1}{\|\mathbf{A}\|_1}. \quad (10)$$

Here $\frac{\|\mathbf{A}^c\|_1}{\|\mathbf{A}\|_1}$ is a regularization term that can control the sparsity of G^c and λ_1 is a hyper-parameter. We enforce the constraint that G^n is unable to predict meaningful information, so its learning target is set as a uniform distribution which push its prediction evenly to all categories. Meanwhile, G^c is used for predicting the true label. This operation can remove some trivial and noisy structure information, which intuitively reduces the difficulty of learning the causal structure in the follow-up phase.

B. Phase 2: Model Update in Output Space

1) *Output-Level Generator*: The first phase described above only captures the interaction between tasks in the feature space, and each task is still separately predicted without modeling structured output [69], [70], [71]. In the second phase, we introduce the output-level generator which is parameterized as a causal EBM to achieve the following two targets: 1) *modeling the joint dependency of task labels to achieve structured prediction*; 2) *refining G^c in the output space to generate the final causal structure*. To finish these goals, we first convert the independent prediction in (5) into the following joint label distribution:

$$p_\theta(\mathbf{y}|\mathbf{x}, G^c) = \frac{e^{-E_\theta(\mathbf{x}, \mathbf{y}, G^c)}}{\Gamma_\theta(\mathbf{x})}, \quad (11)$$

where the energy function $E_\theta(\mathbf{x}, \mathbf{y}, G^c)$ is formalized as:

$$E_\theta(\mathbf{x}, \mathbf{y}, G^c) = - \sum_{i \in V} f_i(\mathbf{x})[y_i] - \sum_{(i,j) \in E^c} f_{ij}(\mathbf{x})[y_i, y_j], \quad (12)$$

where $f_i(\mathbf{x})[y_i]$ represents a first-order scalar factor contributed by the task node i w.r.t. the ground truth label y_i , and $f_{ij}(\mathbf{x})[y_i, y_j]$ represents a second-order scalar factor contributed by the task pair (i, j) in E^c . Through aggregating these two types of output factors in $E_\theta(\mathbf{x}, \mathbf{y}, G^c)$, we can explicitly model G^c in the output space.

$f_i(\mathbf{x})[y_i]$ can be calculated by indexing y_i from $f_i(\mathbf{x})$ in (8). As for $f_{ij}(\mathbf{x})[y_i, y_j]$, we use an edge-level state-aware GNN to estimate it. The input feature of each edge is given as follows,

$$\mathbf{U}_{ij}^0(\mathbf{x}) = \text{MLP}_e^0([k(\mathbf{x})|\mathbf{Z}_i|\mathbf{Z}_j]). \quad (13)$$

The propagation in the l -th layer is defined as:

$$\begin{aligned} \mathbf{U}_{ij}^{l+1}(\mathbf{x})[s, s'] &= \text{MLP}_e^{l+1}(\mathbf{U}_{ij}^l(\mathbf{x})[s, s'] \\ &\quad + \text{MLP}_e^{l+1}(\mathbf{U}_i^l(\mathbf{x})[s] + \mathbf{U}_j^l(\mathbf{x})[s'])), \end{aligned} \quad (14)$$

where the calculations of $\mathbf{U}_i^l(\mathbf{x})[s]$ and $\mathbf{U}_j^l(\mathbf{x})[s']$ follow the computational way in (7) but with separate MLP layers. Similar to $\text{MLP}_c(\cdot)$, both $\text{MLP}_e(\cdot)$ and $\text{MLP}_t(\cdot)$ are two MLP functions. $f_{ij}(\mathbf{x})[s, s']$ is defined as:

$$f_{ij}(\mathbf{x})[s, s'] = \widehat{R}([\mathbf{U}_{ij}^l(\mathbf{x})[s, s'] | l = 1, \dots, L), \quad (15)$$

where $\widehat{R}$ is also a readout function. $f_{ij}(\mathbf{x})[y_i, y_j]$ is calculated by indexing the ground truth label pair (y_i, y_j) , i.e., $s = y_i$ and $s' = y_j$. Additionally, we apply the log operation on both $f_i(\mathbf{x})[y_i]$ and $f_{ij}(\mathbf{x})[y_i, y_j]$ for scaling them to $(0, \infty)$, which is a common choice in training EBM [72].

Structural Equation Model Layer: According to the classic structural equation model (SEM) in causal literature [73], our causal EBM contains the SEM layer to directly describe the effects between tasks driven by causality. Specifically, we first merge the above two types of factors, i.e., $f_i(\mathbf{x})[y_i]$ and $f_{ij}(\mathbf{x})[y_i, y_j]$ into a score vector $\Phi_{\mathbf{x}}(\mathbf{y}) = [\Phi_{\mathbf{x}}(y_1), \dots, \Phi_{\mathbf{x}}(y_m)]$ where $\Phi_{\mathbf{x}}(y_i)$ is defined as:

$$\Phi_{\mathbf{x}}(y_i) = f_i(\mathbf{x})[y_i] + \sum_{j \in N(i)} f_{ij}(\mathbf{x})[y_i, y_j]. \quad (16)$$

Then, we treat $\mathbf{A}^c = [\mathbf{A}_1^c | \dots | \mathbf{A}_m^c]$ as a differentiable variable, and each element $\mathbf{A}_i^c \in \mathbb{R}^m$ represents a weighted column vector where $\mathbf{A}_{ji}^c$ describes the causal strength from $\Phi_{\mathbf{x}}(y_j)$ to $\Phi_{\mathbf{x}}(y_i)$. The linear SEM is described as:

$$\Phi_{\mathbf{x}}(y_i) = \psi\left((\mathbf{A}_i^c)^T \Phi_{\mathbf{x}}(\mathbf{y})\right) + \epsilon_i, \quad (17)$$

where ψ is the ELU activation function [74] used for stable training and ϵ_i denotes an exogenous variable represented as a Gaussian noise. Through this reconstruction mechanism, $\mathbf{A}^c$ can accurately describe how each endogenous score $\Phi_{\mathbf{x}}(y_i)$ is generated by $(\mathbf{A}_i^c)^T \Phi_{\mathbf{x}}(\mathbf{y})$ that contains parental information and filters out its non-parent causes. The reconstruction loss function is

$$\mathcal{L}_{rc} = \sum_{i=1}^m \left\| \Phi_{\mathbf{x}}(y_i) - \psi\left((\mathbf{A}_i^c)^T \Phi_{\mathbf{x}}(\mathbf{y})\right) \right\|^2. \quad (18)$$

Simultaneously, $\mathbf{A}^c$ is constrained to the form of DAG, leading to a continuous differentiable constraint function [61], [63]:

$$\mathcal{L}_{dag} = \text{tr}\left(\left(\frac{1}{m}\mathbf{A}^c \odot \mathbf{A}^c + \mathbf{I}\right)^m\right) - m. \quad (19)$$

When $\mathcal{L}_{dag}$ equals to 0, $\mathbf{A}^c$ is corresponding to a DAG.

Energy Tilting Term: Instead of directly using the joint label distribution defined in (11) to make predictions, we adopt the energy tilting term to help the training of our causal EBM according to previous EBM approaches [20], [72], [75]. Specifically, the exponent of the negative energy function $e^{-E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)}$ is served as a tilting term of an arbitrary pre-trained model $q(\mathbf{y}|\mathbf{x}, G^c)$. This operation reduces the difficulty of training EBMs and yields a new model prediction distribution: $p_{\theta}(\mathbf{y}|\mathbf{x}, G^c) \propto q(\mathbf{y}|\mathbf{x}, G^c)e^{-E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)}$.

2) **Intervention Contrastive Estimation:** EBMs are notoriously difficult to train due to the intractable partition function $\Gamma_{\theta}(\mathbf{x})$. Benefiting from our causal framework, we propose an

intervention contrastive estimation (ICE) to train our causal EBM efficiently.

Theorem 1: Given the (conditional) data label distribution $p_d(\mathbf{y}|\mathbf{x}, G^c)$ and the (conditional) noise label distribution $p_n(\mathbf{y}|\mathbf{x}, G^c)$, in our proposed ICE training the causal EBM can be equivalent to minimizing the following loss function:

$$\begin{aligned} \mathcal{L}_{ice} = & \mathbb{E}_{\mathbf{y} \sim p_n(\mathbf{y}|\mathbf{x}, G^c)} \log\left(1 + e^{-E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)}\right) \\ & + \mathbb{E}_{\mathbf{y} \sim p_d(\mathbf{y}|\mathbf{x}, G^c)} \log\left(1 + e^{E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)}\right). \end{aligned} \quad (20)$$

The theoretical derivation of this loss function is provided here: Let β denote a binary Bernoulli variable, the mixture of noise and data label distributions is given as:

$$\begin{aligned} p_{n,d}(\mathbf{y}|\mathbf{x}, G^c) = & p_n(\mathbf{y}|\mathbf{x}, G^c)p(\beta = 0) \\ & + p_d(\mathbf{y}|\mathbf{x}, G^c)p(\beta = 1). \end{aligned} \quad (21)$$

The posterior probability of $\beta = 0$ is

$$\begin{aligned} p_{\hat{n}, \hat{d}}(\beta = 0|\mathbf{y}) = & \frac{p_{\hat{n}, \hat{d}}(\mathbf{y}|\beta = 0)p(\beta = 0)}{p_{\hat{n}, \hat{d}}(\mathbf{y})} \\ = & \frac{p_{\hat{n}}(\mathbf{y})}{p_{\hat{n}}(\mathbf{y}) + p_{\hat{d}}(\mathbf{y})}, \end{aligned} \quad (22)$$

where $p_{\hat{n}}(\mathbf{y}) = p_n(\mathbf{y}|\mathbf{x}, G^c)$, $p_{\hat{d}}(\mathbf{y}) = p_d(\mathbf{y}|\mathbf{x}, G^c)$ and $p_{\hat{n}, \hat{d}}(\mathbf{y}) = p_{n,d}(\mathbf{y}|\mathbf{x}, G^c)$. Here we assume $\frac{p(\beta=1)}{p(\beta=0)} \equiv 1$. Following the same way, the mixture of noise and model label distributions is

$$\begin{aligned} p_{n,\theta}(\mathbf{y}|\mathbf{x}, G^c) = & p_n(\mathbf{y}|\mathbf{x}, G^c)p(\beta = 0) \\ & + p_{\theta}(\mathbf{y}|\mathbf{x}, G^c)p(\beta = 1). \end{aligned} \quad (23)$$

The posterior probability of $\beta = 0$ is then

$$\begin{aligned} p_{\hat{n}, \hat{\theta}}(\beta = 0|\mathbf{y}) = & \frac{p_{\hat{n}, \hat{\theta}}(\mathbf{y}|\beta = 0)p(\beta = 0)}{p_{\hat{n}, \hat{\theta}}(\mathbf{y})} \\ = & \frac{p_{\hat{n}}(\mathbf{y})}{p_{\hat{n}}(\mathbf{y}) + p_{\hat{\theta}}(\mathbf{y})}, \end{aligned} \quad (24)$$

where $p_{\hat{\theta}}(\mathbf{y}) = p_{\theta}(\mathbf{y}|\mathbf{x}, G^c)$ and $p_{\hat{n}, \hat{\theta}}(\mathbf{y}) = p_{n,\theta}(\mathbf{y}|\mathbf{x}, G^c)$. The optimal model parameter θ^* can be obtained by minimizing the discrepancy between $p_{\hat{n}, \hat{d}}(\beta|\mathbf{y})$ and $p_{\hat{n}, \hat{\theta}}(\beta|\mathbf{y})$:

$$\begin{aligned} \theta^* = & \arg\min_{\theta} \mathbb{E}_{p_{\hat{n}, \hat{d}}(\mathbf{y})} \text{KL}\left(p_{\hat{n}, \hat{d}}(\beta|\mathbf{y}) || p_{\hat{n}, \hat{\theta}}(\beta|\mathbf{y})\right) \\ = & \arg\min_{\theta} \mathbb{E}_{p_{\hat{n}, \hat{d}}(\mathbf{y}, \beta)} \left[\log p_{\hat{n}, \hat{d}}(\beta|\mathbf{y}) - \log p_{\hat{n}, \hat{\theta}}(\beta|\mathbf{y}) \right] \\ = & \arg\max_{\theta} \mathbb{E}_{p_{\hat{n}, \hat{d}}(\mathbf{y}, \beta)} \log p_{\hat{n}, \hat{\theta}}(\beta|\mathbf{y}) \\ = & \arg\max_{\theta} \mathbb{E}_{p_{\hat{n}, \hat{d}}(\mathbf{y}, \beta)} \left[\log p_{\hat{n}, \hat{\theta}}(\beta = 0|\mathbf{y}) \right. \\ & \left. + \log p_{\hat{n}, \hat{\theta}}(\beta = 1|\mathbf{y}) \right] \\ = & \arg\max_{\theta} \mathbb{E}_{p_{\hat{n}, \hat{d}}(\mathbf{y}, \beta)} \left[\log \frac{p_{\hat{n}}(\mathbf{y})}{p_{\hat{n}}(\mathbf{y}) + p_{\hat{\theta}}(\mathbf{y})} \right. \\ & \left. + \log \frac{p_{\hat{\theta}}(\mathbf{y})}{p_{\hat{n}}(\mathbf{y}) + p_{\hat{\theta}}(\mathbf{y})} \right] \end{aligned}$$

$$\begin{aligned}
&= \operatorname{argmax}_{\theta} \mathbb{E}_{p_{\hat{n}, \hat{d}}(\mathbf{y}, \beta)} \left[\log \frac{p_n(\mathbf{y}|\mathbf{x}, G^c)}{p_n(\mathbf{y}|\mathbf{x}, G^c) + p_{\theta}(\mathbf{y}|\mathbf{x}, G^c)} \right. \\
&\quad \left. + \log \frac{p_{\theta}(\mathbf{y}|\mathbf{x}, G^c)}{p_n(\mathbf{y}|\mathbf{x}, G^c) + p_{\theta}(\mathbf{y}|\mathbf{x}, G^c)} \right] \\
&= \operatorname{argmax}_{\theta} \mathbb{E}_{p_{\hat{n}, \hat{d}}(\mathbf{y}, \beta)} \left[\log \frac{p_n(\mathbf{y}|\mathbf{x}, G^c)}{p_n(\mathbf{y}|\mathbf{x}, G^c) + p_n(\mathbf{y}|\mathbf{x}, G^c)e^{-E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)}} \right. \\
&\quad \left. + \log \frac{p_n(\mathbf{y}|\mathbf{x}, G^c)e^{-E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)}}{p_n(\mathbf{y}|\mathbf{x}, G^c) + p_n(\mathbf{y}|\mathbf{x}, G^c)e^{-E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)}} \right] \\
&= \operatorname{argmin}_{\theta} \left[\mathbb{E}_{p_{\hat{n}}(\mathbf{y}, \beta)} \log \left(1 + e^{-E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)} \right) \right. \\
&\quad \left. + \mathbb{E}_{p_{\hat{d}}(\mathbf{y}, \beta)} \log \left(1 + e^{E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)} \right) \right] \\
&= \operatorname{argmin}_{\theta} \mathcal{L}_{ice}. \tag{25}
\end{aligned}$$

Here $p_{\theta}(\mathbf{y}|\mathbf{x}, G^c)$ is defined in Section 1 and $q(\mathbf{y}|\mathbf{x}, G^c)$ is acted as the noise distribution for simplifying calculations. Put $p_{\hat{n}, \hat{\theta}}(\beta = 0|\mathbf{y})$, $p_{\hat{n}, \hat{\theta}}(\beta = 1|\mathbf{y})$ and $\Gamma_{\theta}(\mathbf{x}) = 1$ into (25), and the proof of Theorem 1 is completed.

According to the principle of noise contrastive estimation (NCE) [76], how to design a suitable noise distribution is a critical step to train EBMs. We perform the intervention on $\Phi_{\mathbf{x}}(y_i)$ to calculate $p_n(\mathbf{y}|\mathbf{x}, G^c)$. Intervention [77] means that we can intentionally assign some values to a certain part of a system and obtain some interesting outcomes of such manipulations. Concretely, we can set $\Phi_{\mathbf{x}}(y_i)$ on the RHS of (17) to a fixed value for delivering its effect to all children nodes on the LHS of (17). By manipulating this causal framework, we can estimate the noise energy function $E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)$ in $p_n(\mathbf{y}|\mathbf{x}, G^c)$ to generate the counterfactual negative samples for model training. Based on the above description, the whole loss function of the second phase can be defined as

$$\mathcal{L}_e = \mathcal{L}_{ice} + \lambda_2 \mathcal{L}_{rc} + \lambda_3 \mathcal{L}_{dag}, \tag{26}$$

where λ_2 and λ_3 are also two hyper-parameters.

C. Training and Inference

GCNet follows a two-phase training mode. In the first phase, we try to learn the structure prior G^c in the feature space by optimizing the objective function in (10). In the second phase, G^c is further refined to generate the causal structure by minimizing the loss function $\mathcal{L}_e$ in the output space. The pseudo-code of training procedure of GCNet is provided in Algorithm 1. We adopt Adam [78] to optimize model parameters.

In the process of inference, we calculate the marginal distribution of each task for label prediction. Gibbs sampling [20], [68], [79] is utilized to approximate this distribution. It is a well-known Markov chain Monte Carlo (MCMC) algorithm for generating samples by sweeping through each variable to sample from its conditional distribution with the remaining variables fixed to their current values. In our implementation, for each input with m task labels, i.e., $(\mathbf{x}, \{y_i\}_{i=1}^m)$, we can sample the i -th task label and keep other remaining task labels $\mathbf{y}_{-i}$ fixed.

Algorithm 1: The training procedure of GCNet.

Require: The data distribution $p_d(\mathbf{x}, \mathbf{y})$; the constructed task relation graph $G = \{V, E\}$ with $\mathbf{A}$.
Ensure: The trained feature-level generator, the initial input and task encoders, the node-level and edge-level state-aware GNNs and the causal task relation graph $G^c = \{V, E^c\}$ with $\mathbf{A}^c$.
1: Initialize all model parameters.
2: **#Model update in feature space:**
3: **for** sampled $\mathbf{x}$ and $\mathbf{y}$ **do**
4: Calculate initial embeddings, i.e., $k(\mathbf{x})$ and $\{\mathbf{Z}_i\}_{i=1}^m$;
5: Generate the confident matrix $\mathbf{C}$ via Eq. (2)–(3);
6: Obtain G^c and G^n via Eq. (4);
7: Estimate $p(\mathbf{y}|\mathbf{x}, G^c)$ and $p(\tilde{\mathbf{y}}|\mathbf{x}, G^n)$ with the independent assumption via Eq. (6)–(9);
8: Maximize the log-likelihood of the joint label distribution in Eq. (10) by the Adam optimizer.
9: **end for**
10: **#Model update in output space:**
11: **for** sampled $\mathbf{x}$ and $\mathbf{y}$ **do**
12: Calculate initial embeddings, i.e., $k(\mathbf{x})$ and $\{\mathbf{Z}_i\}_{i=1}^m$;
13: Calculate $E_{\theta}(\mathbf{x}, \mathbf{y}, G^c)$ in Eq. (12) via Eq.(13)–(15);
14: Refine G^c via minimizing the loss functions in Eq. (18)–(19);
15: Minimize the ICE loss in Eq. (20) by the Adam optimizer.
16: **end for**

The update function of each iteration is:

$$p_{\theta}(y_i|\mathbf{x}, \mathbf{y}_{-i}, G^c) = \frac{e^{f_i(\mathbf{x})[y_i] + \sum_{(i,j) \in E^c} f_{ij}(\mathbf{x})[y_i, y_j]}}{\sum_{y_i=1}^{o-1} e^{f_i(\mathbf{x})[y_i] + \sum_{(i,j) \in E^c} f_{ij}(\mathbf{x})[y_i, y_j]}}. \tag{27}$$

In addition, both phases of model updates are necessary during the testing stage, because the node-level state-aware GNN from the first phase is also required for calculating the first-order scalar factor.

D. Time Complexity

In the first phase, GCNet includes two calculation procedures. The first procedure is the feature-level generator which uses GCN to estimate the confidence matrix and derive the informative and non-informative structure priors. The time complexity of this part is $O(|E|)$ where E is the edge set of the task relation graph. The second procedure is the independent prediction which approximates the true-label and pseudo-label distributions. The time complexities of these two approximations are also $O(|E|)$, so the total time complexity of the first phase is $O(|E|)$.

In the second phase, GCNet uses the edge-level state-aware GNN to calculate the energy function, the time complexity of this part is $O(|E|)$. The calculation of linear SEM in (17)–(18) would cost $O(|V|^2)$ where V is the node set of the task relation graph. For the DAG constraint, we use the following ways to simplify calculations: 1) using the regularization term in (10) to

generate a sparse structure of the causal graph; 2) making the first-order approximation of (19), both of which are commonly used tricks in causal literature [61], [77], [80]. In fact, regarding SEM updates, there exist other representative approaches, such as constraint-based and score-based methods [81], [82]. Their computational complexity is typically higher than ours, usually $O(|V|^k)$, where k denotes the maximum number of neighbors or the maximum in-degree in a graph. In contrast, our method adopts a continuous optimization manner and only requires a single iteration, thereby reducing the complexity.

Therefore, the training time complexity and the test time complexity of our model are $O(|E| + |V|^2)$ and $O(|E|)$. In Section I, we provide the concrete runtime comparison between our model and multiple strong baselines.

Here, we further highlight the distinctions between GCNet and existing MTL methods. 1) Rather than assuming the task relation graph to be reliable, GCNet performs causal structure learning on this graph to uncover the true causal relationships among tasks, thereby boosting MTL performance. 2) We design a new two-phase model update: the first phase focuses on learning the structure prior in the feature space, while the second phase refines this prior to learn the causal structure in the output space. 3) We introduce the causal EBM to achieve the model update in the second phase and further propose ICE to train this energy-based generative model effectively.

V. EXPERIMENT

In this section, we seek to answer the following major research questions (RQs):

- **RQ1:** Does our method achieve the supreme performance in comparison with multiple strong MTL approaches?
- **RQ2:** Can the learned causal task relation graph remove some spurious edges existed in original task relation graph and discover some true causal relations between tasks?
- **RQ3:** Is the proposed ICE strategy more effective than other NCE heuristics?
- **RQ4:** GCNet includes two-phase learning on the both feature and output spaces. Are both of them helpful to improve model performance?
- **RQ5:** How sensitive is our model with respect to different hyper-parameters?
- **RQ6:** Besides the theoretical analysis of time complexity, What is the empirical runtime of our model and compared baselines?

To answer these questions, we do a detailed comparative analysis of our model. For experiment reproducibility, we provide the source code at <https://github.com/Note2Q/GCNet>.

A. Datasets

We evaluate our model on four types of datasets. For each dataset, the task is treated as a binary classification. The division ratio of training, validation and test sets is 8:1:1. The data statistics is shown in Table II. Remarkably, School, ChEMBL 10, ChEMBL 50 and ChEMBL 100 contain hundreds of tasks, so the scales of task numbers in our experiments are sufficient in MTL settings [12].

TABLE II
STATISTICS OF DATASETS. ‘SPARSITY’ DENOTES THE GRAPH DENSITY OF TASK RELATION GRAPH

Dataset	#Inputs	#Tasks	#Edges	Sparsity
Synth	500	50	203	8.12%
Sushi	5,000	10	24	24.00%
School	15,362	139	862	4.46%
ChEMBL 10	13,004	382	3,571	2.45%
ChEMBL 50	932	152	790	3.42%
ChEMBL 100	518	132	604	3.47%

- **Synth** represents a synthetic dataset which is created by ourselves according to previous works [22], [24]. Synth includes 50 tasks with 203 edges between them. The numbers of input data is 500, and the dimension size of the initial feature vector is 20. As suggested by [24], we adopt multiple cluster centers to generate initial feature vectors where each feature is drawn from a different clustering scheme. We use the Stochastic Block Model (SBM) [83] to generate a task relation graph as the ground truth and add some noise edges into this graph as the spurious edges.
- **Sushi** is a user preference prediction dataset in sushi (a Japanese food) [84]. This dataset contains the results of a series of surveys conducted asking 5000 individuals for their preferences about 10 different types of sushi. We transformed the original ranking data into binary format, where each task is to predict whether users like or dislike specific sushi items. The input consists of 11 user-specific features and 9 item-specific features. We construct the task relation graph for Sushi based on the co-occurrence of input features related to sushi items. First, each sushi item is represented as a node in the task relation graph. Then, for each pair of sushi items, we count the co-occurrence frequencies of 9 item-specific features and assign these normalized frequencies as weights to the edges in the task relation graph.
- **School** is a benchmark MTL dataset for predicting the score level for students [12]. School dataset is to deal with the prediction of student test results for a collection of schools, based on school demographics [85]. It comprises examination records of 15,362 students from 139 secondary schools in London for the years 1985, 1986, and 1987. Therefore, there are 139 tasks, each focused on predicting student performance within a specific school. We convert the original regression problem as the classification problem. Each task involves predicting whether students’ exam scores are considered good or not for a specific school. The input consists of four school-specific and three student-specific attributes. The task relation graph for the school is constructed in the same manner with Sushi, which is based on the co-occurrence of school-related input features.
- **ChEMBL 10, ChEMBL 50 and ChEMBL 100** are three drug discovery datasets for evaluating the performance of solving the problem of molecular property prediction which has been a standard application of MTL [67]. They are divided into different datasets according to the different thresholds of non-missing labels, since some of

the molecule-property labels may be missing [20]. Each input data is a molecule that can be represented as a molecular graph with corresponding atoms and bonds. The task relation graph is also constructed by SGNN [20]. Concretely, since all tasks are protein-targeting, so the protein-protein interaction (PPI) data can be used for extracting task relations. The authors combine a chemical database of bioactive molecules ChEMBL [86] with the protein-protein interaction graph STRING [25]. For each task, they first collect UniProt IDs of the targeted proteins and combine all of them into a UniProt ID set. Then, they query STRING to obtain PPI scores of all pairs of proteins in this set. Finally, a heuristic rule, i.e., max operation is conducted on this set to define the relation between two tasks.

From the above description, we can see that the task relation of all datasets is approximated by using some heuristic rules instead of the causal relation.

B. Experimental Setup

1) *Evaluation Metrics*: Following previous works on MTL [20], [67], we adopt the mean of Receiver Operating Characteristic-Area Under the Curve (ROC-AUC) over all tasks as the evaluation metric. Notice that a higher value reflects a better result. To verify whether the results are statistically significant, we run 5 random seeds and the *mean $\pm$ standard deviation* is reported.

2) *Compared Baselines*: We compare our method with the following baselines:

- Standard single-task learning (SSTL) [67]. SSTL adopts MLPs to calculate the loss function of each task separately. Hence, it is the most basic baseline without considering any task relations.
- Standard multi-task learning (SMTL) [12]. Different from SSTL, SMTL summarizes the loss function of all tasks together for updating model parameters.
- Multiple-gradient descent algorithm (MGDA) [87]. MGDA reformulates MTL as multi-objective optimization, with the goal of finding a Pareto optimal solution. MGDA is compatible with DNNs since it can use the gradient-based optimization and converge to a point on the Pareto set provably.
- Uncertainty-aware multi-task learning (UMTL) [88]. UMTL models the homoscedastic uncertainty to weigh the loss function of each task. It designs a loss function which can learn a relative weighting from data and is robust to weight initialization.
- Gradient normalization (GradNorm) [89]. The imbalanced gradients across tasks would result in sub-optimal results within a MTL framework. GradNorm includes a way of gradient normalization that can dynamically adjust gradient magnitudes to balance different tasks.
- Multi-gate mixture-of-experts (MMoE) [29]. MMoE contains the light-weight modulation and gating networks to automatically adjust parameterization between modeling

shared information and modeling task-specific information.

- Multi-task attention network (MTAN) [90]. MTAN includes a shared DNN and multiple task-specific attention DNNs to learn the task-shared and task-specific features. The shared network contains a global feature pool, while each attention module applies a soft attention mask to the shared DNN to learn task-specific features.
- Loss-balanced task weighting (LBTW) [91]. Although the results may improve on average over all tasks, the performance of MTL methods can be worse than a single-task model. LBTW learns task-specific weights to reduce the effect of negative transfer.
- State graph neural network (SGNN) [20]. SGNN adopts a message-passing framework to learn task representations from the constructed task relation graph. But it does not consider the joint dependency of task labels.
- State graph neural network with the EBM (SGNN-EBM) [20]. SGNN-EBM overcomes the drawback of independent prediction in SGNN by using the EBM to model the task relation graph in the output space.

C. Reproducible Configuration

1) *Running Environment*: The experiments are conducted on a Linux server with Intel(R) Xeon(R) CPU E5-2620 v4 @ 2.10 GHz, 128 G RAM and NVIDIA Tesla V100. The operating system is Ubuntu 18.04. We implement our model and all baselines with the deep learning library PyTorch (version 1.6) in Python 3.7.

2) *Encoder Implementation*: In our method, we adopt a three-layer MLP to encode the input data for synthetic dataset. For real-world datasets, we adopt GIN [48] to generate molecule embeddings. Specifically, for a given molecular graph, i.e., $\mathbf{x} = (\mathbf{F}^a, \mathbf{F}^b)$ where $\mathbf{F}^a$ and $\mathbf{F}^b$ represent the feature matrices of atom types and bond types respectively. We assume a molecule owns η atoms with ξ bonds. The embedding update in GIN for the i -th atom is given as:

$$k_i^{l+1}(\mathbf{x}) = \text{MLP}_a^{l+1} \left(k_i^l(\mathbf{x}) + \sum_{j \in N(i)} k_j^l(\mathbf{x}) + \text{MLP}_b^{l+1}(\mathbf{F}_{ij}^b) \right), \quad (28)$$

where both MLP_a^{l+1} and MLP_b^{l+1} are three-layer MLPs for encoding atoms and bonds in the $l+1$ -th layer respectively. $\mathbf{F}^a$ is used for initial atom representations. The layer number of GIN L' is fixed as 5, and the outputs of last layer are taken as the final atom representations. The input embedding $k(\mathbf{x})$ is the mean of all atom representations:

$$k(\mathbf{x}) = \frac{1}{\eta} \sum_{i=1}^{\eta} k_i^{L'}(\mathbf{x}). \quad (29)$$

The initial task representations in all datasets are obtained via a two-layer GCN. Each initial task attribute is a one-hot vector.

For all baselines, we use the same encoders as ours to encode the input data and task representations. For SSTL, SMTL, MGDA, UMTL, GradNorm, MTAN and LBTW, we use a

TABLE III
RESULTS ON ALL DATASETS. FROM THIS TABLE, WE CAN CONCLUDE THAT OUR MODEL ACHIEVES CONSISTENT IMPROVEMENTS OVER ALL COMPARED BASELINES

Method	Synth	Sushi	School	CHEMBL 10	CHEMBL 50	CHEMBL 100
SSTL	76.92±1.30	80.19±1.30	74.70±1.45	71.67±0.64	73.57±1.20	70.81±1.28
SMTL	78.22±1.56	81.37±1.42	76.44±1.71	74.84±0.61	79.37±1.76	77.78±1.59
MGDA	79.02±1.15	81.85±1.31	77.28±1.52	74.79±0.57	79.53±1.10	78.37±1.48
UMTL	80.07±0.96	82.04±1.12	77.35±1.06	72.49±0.53	79.68±0.98	78.71±1.93
GradNorm	80.37±1.72	82.49±1.63	78.28±1.29	75.17±0.77	79.46±1.27	78.75±1.60
MMoE	80.42±0.85	83.89±0.94	78.83±1.09	74.65±0.68	79.38±0.76	78.70±0.65
MTAN	79.94±2.15	82.68±1.59	77.90±1.73	72.45±1.31	79.35±0.68	78.21±2.31
LBTW	80.10±1.18	83.37±1.55	78.52±0.76	75.21±0.49	79.52±0.56	79.07±0.99
SGNN	80.48±1.02	83.92±0.97	79.03±0.83	77.90±0.88	79.67±0.87	80.19±0.67
SGNN-EBM	81.78±1.02	84.74±0.89	80.16±1.02	78.35±1.07	80.54±1.02	81.15±0.59
GCNet	84.79±0.72	86.83±0.67	82.29±0.79	80.28±0.57	82.46±0.54	82.87±0.72

three-layer MLP to predict task labels. For MMoE, the gating networks, experts and tower networks are respectively implemented by the linear transformations with the softmax function, three-layer MLPs and single-layer networks according to the original paper. For SGNN, we use the two-layer SGNN to propagate neighbor information and a three-layer MLP to predict each task independently.

3) *Hyper-Parameter Setting*: The grid search is used to find the optimal values of hyper-parameters. In particular, the harmonic factors used in the loss functions, i.e., λ_1 , λ_2 and λ_3 are uniformly selected from {0.01, 0.05, 0.1, 0.5, 1.0}. The search intervals of batchsize and of the layer number of GNNs L are {8, 16, 32, 64, 128} and {1, 2, 3, 4} respectively. The learning rate is fixed as $1e-3$ for all datasets. GCNet is iterated with 300 epochs in the first and second training phases, respectively. According to the guidance of literature, compared baselines are also fully trained for model convergence.

For fair comparisons, all baselines and our method use the same dimension size of both the input embedding and the task embedding. The dimension size of the task embedding is fixed as 100. The dimension sizes of the input embedding for different datasets are listed here: 20 for Synth, 100 for Sushi, School and ChEMBL 10, and 50 for both ChEMBL 50 and ChEMBL 100.

The concrete settings of all hyper-parameters in our model are listed here: {0.1, 0.1, 0.1, 64, 2} for Synth; {0.1, 0.05, 0.1, 64, 2} for Sushi; {0.1, 0.05, 0.05, 64, 2} for School; {0.1, 0.05, 0.05, 16, 2} for ChEMBL 10; {0.5, 0.1, 0.1, 64, 2} for both ChEMBL 50 and ChEMBL 100.

D. Performance Comparison (A1)

Table III demonstrates the performances of all models on synthetic and real-world datasets. The best performances are highlighted in bold. From Table III, we can draw the following conclusions:

- GCNet consistently yields the best performances on all datasets. Compared with SOTA baselines, the most obvious improvement is that GCNet brings 3.7% improvement on Synth. It demonstrates that the constructed task relation graph is not entirely trustworthy, and learning how to find the true causal structure between tasks is important.
- It is obvious that SMTL methods achieve better results than STL and MTL baselines. Therefore, learning the task relations through using the task relation graph is helpful to improve model performance.

- In general, jointly learning all tasks can effectively alleviate the data sparsity problem, because MTL methods have a distinct advantage over STL in practice.

E. Visualization Analysis (A2)

We provide visual experiments on learning the causal structure from the constructed task relation graph. Fig. 3 demonstrates the empirical result on ChEMBL 100. Fig. 3 includes six serine/threonine kinases: **DAPK3**, **CDS1**, **PKCG**, **PI3K**, **MSK1** and **ERK2** which are randomly selected (marked as 0-5). Each task is to infer the activation between the given kinase and molecule (active or inactive). The actual effect of these kinases is listed here: DAPK3 mainly regulates apoptosis and transcription; CDS1 is required for cell cycle arrest and activation of DNA repair; PKCG plays diverse roles in sensitivity to pain, alcohol and inhibition of stimulation; PI3K inhibits the apoptotic pathway in cells for promoting cell survival; MSK1 phosphorylates transcription factors in the cell nucleus to influence gene expression; ERK2 regulates transcription upon stimulation.

In left-to-right layout of Fig. 3, (a) is the constructed task relation graph; (b) is the learned structure prior; (c) is the causal structure of task relations in the output space; (d) is the ground truth derived from the augmented Lagrangian method with the test labels. From Fig. 3, we have the following conclusions:

- Compared with (a), the learned structure prior is useful. For example, as shown in (d) of Fig. 3, there is an edge from DAPK3 (0) to ERK2 (5), because when cells become apoptotic, ERK2 will be required to take part in this procedure through the regulation of DAPK3. Therefore, the structure prior can strengthen the causal edges between these two tasks, but it is still very different from the true one globally.
- The refinement of G^c in the output space is essential to generate the final causal structure. It can remove some inverse negative effects, such as ERK2 (5) to CDS1 (1) in (a) of Fig. 3, and find some actual causes at the same time. For example, there is a true causal edge from PKCG (2) to ERK2 (5) shown in (d) of Fig. 3, because PKCG can directly phosphorylate ERK2, thereby activating ERK2 to regulate cell transcription under stimulation. The update in the output space can learn this causal edge and is closer to the ground truth, which verifies the effectiveness of our causal EBM.

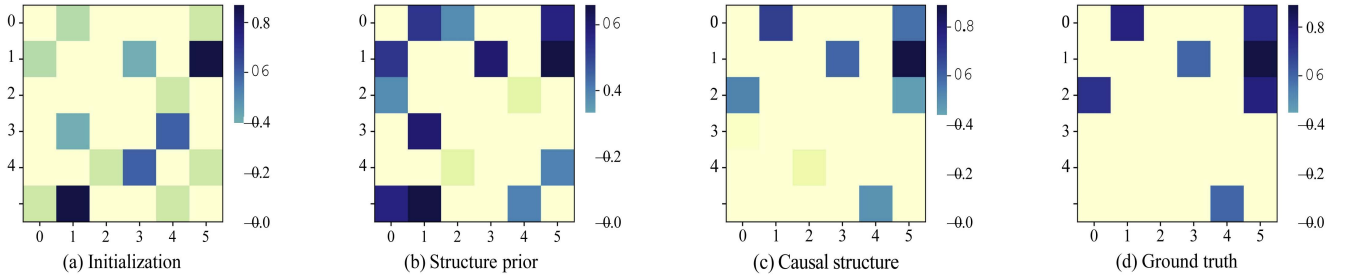


Fig. 3. Learned task relation structures in different phases for the real-world dataset CHEMBL 100.

TABLE IV

RESULTS OF DIFFERENT NOISE DISTRIBUTIONS ON ALL DATASETS. FROM THIS TABLE, WE CAN CONCLUDE THAT THE PROPOSED ICE IS THE MOST EFFECTIVE STRATEGY COMPARED WITH OTHER NOISE DISTRIBUTIONS

noise distribution	Synth	Sushi	School	CHEMBL 10	CHEMBL 50	CHEMBL 100
Uniform	80.12±1.73	82.26±1.54	75.49±2.71	73.92±3.89	74.36±1.02	76.80±1.68
SMTL	81.16±0.86	82.73±1.03	76.73±1.80	76.68±0.58	80.87±1.27	79.47±1.70
UMTL	80.57±0.78	83.98±1.21	76.92±1.76	75.88±0.93	80.98±0.86	80.16±1.42
MGDA	80.46±0.93	83.34±1.29	77.85±1.52	75.14±1.35	79.57±0.73	79.82±1.36
UMTL	80.57±0.78	84.66±1.70	78.16±2.04	75.88±0.93	80.98±0.86	80.16±1.42
GrandNorm	81.62±1.06	84.10±1.35	77.51±1.55	76.78±0.75	80.28±0.86	80.54±1.36
MMoE	81.03±0.98	85.08±0.97	76.93±1.70	76.42±0.80	79.41±0.84	80.36±1.12
MTAN	81.27±1.12	84.89±1.86	78.64±2.35	76.22±0.63	79.05±0.69	80.28±1.04
LBTW	82.23±0.85	85.14±1.62	78.43±2.16	77.26±0.82	80.10±0.73	80.75±0.86
SGNN	83.02±0.96	85.60±1.18	80.37±1.21	78.62±1.06	80.78±0.83	81.20±0.68
ICE (ours)	84.79±0.72	86.83±0.67	82.29±0.79	80.28±0.57	82.46±0.54	82.87±0.72

F. NCE Choice (A3)

Choosing a suitable noise distribution p_n as the NCE strategy is important to the success of training EBMs. To verify the effectiveness of the proposed ICE, we try to multiple noise distributions including the uniform label distribution and some pre-trained baselines introduced in Section 2 for training EBMs. The empirical results of these NCE strategies are shown in Table IV. From it, we can make the following observations:

- The noise distribution plays an important role of training EBMs. The non-informational uniform distribution would largely undermine model performance. In contrast, ICE is consistent with our causal EBM, and it achieves the best empirical results without any complicated operations. Compared with the performance of uniform distribution, the most obvious improvement is that GCNet achieves 10.9% result lifts on CHEMBL 50.
- The pre-trained SGNN is also a good option. When we use SGNN as the noise distribution, our method is still better than all baselines listed in Table III. Compared with SGNN-EBM shown in Table III, GCNet armed with the noise distribution of SGNN provides 1.5% improvements on Synth.
- Compared with the results on synthetic dataset, the fluctuating ranges of model performances on real-world datasets are larger. So selecting an appropriate noise distribution can better help EBMs to model the more complex data distributions.

G. Ablation Study (A4)

Our method includes a two-phase model update. The first phase of model update is the edge-level generator that aims to

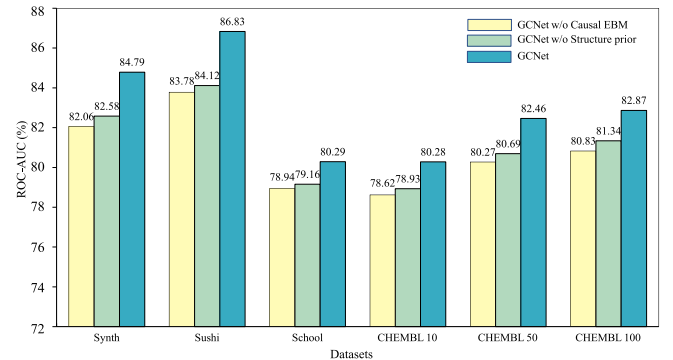


Fig. 4. Ablation study of GCNet. From this figure, we can conclude that the learned structure prior and our causal EBM are critical to improve model performance.

learn G^c in the feature space; the second phase of model update is the proposed causal EBM that refines G^c in the output space. In this section, we conduct the ablation study to show their individual effects on all datasets. ‘GCNet w/o Causal EBM’ denotes the one only performs the model update of the first phase, and ‘GCNet w/o Structure prior’ denotes the one that removes the first phase and directly learns the causal structure in the output space. Fig. 4 presents the experimental results. From it, we have the following observations:

- The improvements of the causal EBM on all datasets are obvious. The most obvious improvement lies in that GCNet brings 3.6% improvement on Sushi. So refining G^c in the output space for finding the causal structure is particularly important.
- The results of ‘GCNet w/o Structure prior’ show a decline compared to ‘GCNet’, demonstrating that the structure

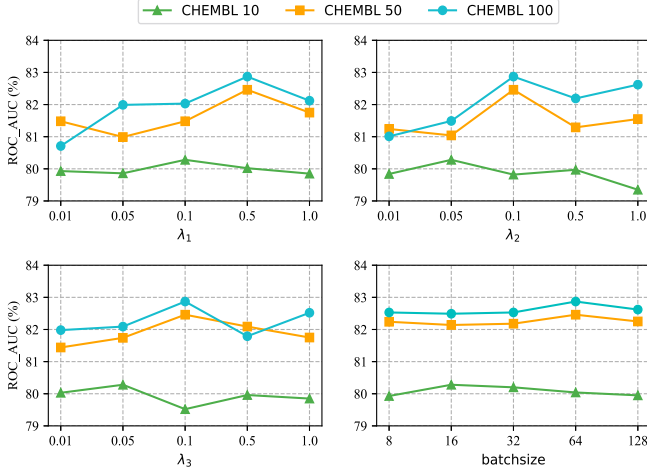


Fig. 5. Hyper-parameter sensitivity of GCNet with regards to λ_1 , λ_2 , λ_3 and batchsize.

prior learned in the feature space can reduce the difficulty of learning the causal structure and effectively promote model performance. The most significant improvement is that GCNet brings 3.2% improvement on Sushi.

H. Hyper-Parameter Sensitivity (A5)

In this section, we first study the hyper-parameter sensitivity of our model in terms of three harmonic factors in the loss functions, i.e., $\{\lambda_1, \lambda_2, \lambda_3\}$ and batchsize. Experiments are conducted on three drug discovery datasets. The corresponding results are provided in Fig. 5, from which we can observe that:

- Compared with CHEMBL 50 and CHEMBL 100, the performance of our model on CHEMBL 10 is more stable. We suppose the main reason is that the amount of data in CHEMBL 10 is larger than CHEMBL 50 and CHEMBL 100, so more data samples can help our model to be insensitive to the changes of these hyper-parameters.
- In most cases, GCNet is better than compared baselines shown in Table III, even in the worst settings of these hyper-parameters. Our model is therefore robust to hyper-parameter selection.
- Compared with λ_1 , λ_2 and λ_3 , the model performance is actually less affected by the change of batchsize. Therefore, To help the model converge more quickly, a relatively large value can be chosen.

In the above experiment, when we tune the optimal value of one hyper-parameter, the other hyper-parameters are fixed at their optimal values. Due to the fact that λ_2 and λ_3 appear together in (26), so we study the joint effect of them for model performance. We conduct the experiment on four datasets and the concrete result is shown in Fig. 6. From it, we can have the following conclusions:

- Compared with λ_3 , the influence of λ_2 is more obvious. It is a reasonable observation, since λ_2 is used for controlling the effect of $\mathcal{L}_{rc}$ which is actually more important than the DAG constraint represented by $\mathcal{L}_{dag}$.

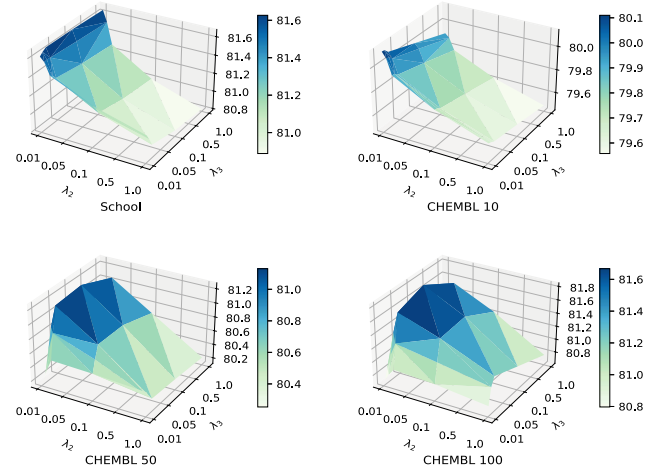


Fig. 6. The joint effect of λ_2 and λ_3 .

TABLE V
THE EFFECT OF THE NUMBER OF LAYERS

#Layer	CHEMBL 10	CHEMBL 50	CHEMBL 100
1	79.74 $\pm$ 0.72	82.18 $\pm$ 0.64	82.34 $\pm$ 0.79
2	80.28$\pm$0.57	82.46$\pm$0.54	82.87$\pm$0.72
3	79.93 $\pm$ 0.68	82.23 $\pm$ 0.71	82.36 $\pm$ 0.83
4	79.87 $\pm$ 0.65	82.12 $\pm$ 0.69	82.41 $\pm$ 0.76
5	79.82 $\pm$ 0.70	81.97 $\pm$ 0.75	82.33 $\pm$ 0.82

- GCNet is robust to the joint variation of λ_2 and λ_3 , because we can see that the results show small fluctuations for these datasets. In contrast, the performance of School and CHEMBL 10 is more stable, while CHEMBL 50 and CHEMBL 100 exhibit greater fluctuations. We suppose this may be due to the label sparsity of CHEMBL 50 and CHEMBL 100.

We also investigate the effect of the number of layers L for both node-level and edge-level state-aware GNNs. L is selected from 1 to 5 and other hyper-parameters are fixed. The experiment is conducted on three real-world datasets and the concrete result is shown in Table V. From it, we have the following conclusions:

- When L is set as 2, our model achieves the best results on these datasets. In light of the fact that it enables our model to better capture the long-term dependence between nodes of the same class, a slight increase in the number of GNN layers thus improves model performance.
- Blindly stacking model layers would increase the risk of model over-fitting, which would be harmful to our model. There may be an over-smoothing phenomenon in our GNNs because our model only uses the first-order and second-order factors in the energy function.

I. Runtime Comparison (A6)

Besides the theoretical analysis of time complexity in Section D, we also conduct the empirical runtime comparison between our model and several MTL baselines in terms of training time and test time. The experiment is conducted on CHEMBL 10 which owns the maximum number of tasks. Fig. 7 shows the

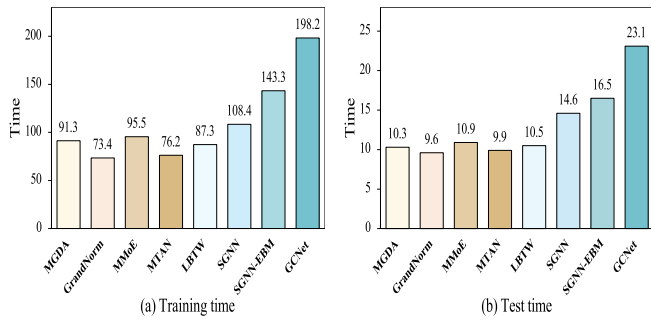


Fig. 7. Runtime comparison (in seconds).

practical training time of an entire epoch and the total test time. From Fig. 7, we can conclude that GCNNet keeps an acceptable computational complexity for both training and test compared with multiple strong MTL baselines. Hence, our model keeps an acceptable time complexity and achieves a good trade-off between model effectiveness and model efficiency.

VI. CONCLUSION AND FUTURE WORK

In this paper, we present GCNNet, a novel generative framework that can reveal the causal structure among tasks to further improve the generalization ability and model robustness of current works. GCNNet includes a two-phase model update. Specifically, GCNNet first contains a feature-level generator to recognize the informative and non-informative structure priors from the constructed task relation graph by approximating the true-label and pseudo-label distributions respectively. Then an output-level generator is introduced to refine the learned structure prior to generate the final causal structure via the linear SEM layer in the output space. By utilizing the causal framework, we theoretically derive an efficient way of training the proposed causal EBM in a manner of contrastive learning. Extensive experimental results and detailed analyses demonstrate that GCNNet outperforms several competitive MTL approaches consistently.

The following directions are worth investigating in future: 1) Our model exploits GNN-based frameworks to learn the task interactions in the feature and output spaces. Consequently, our model is vulnerable to the common issues of GNNs, such as over-smoothing [92] and over-squashing problems [93]. How to design more sophisticated GNNs to alleviate these problems for better learning the energy factors in causal EBM is an interesting direction. 2) GCNNet is now only suitable to solve supervised learning tasks. How to apply our model to other types of tasks, such as active learning and semi-supervised learning, is also an important future step. 3) The used intervention strategy employed in the proposed ICE is heuristic. Exploring more efficient intervention strategies to generate high-quality negative samples is another promising research direction.

REFERENCES

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

- [2] H.-T. Cheng et al., "Wide & deep learning for recommender systems," in *Proc. 1st Workshop Deep Learn. Recommender Syst.*, 2016, pp. 7–10.
- [3] G. Jeh and J. Widom, "Scaling personalized web search," in *Proc. 12th Int. Conf. World Wide Web*, 2003, pp. 271–279.
- [4] K. Muhammad et al., "Deep learning for safe autonomous driving: Current challenges and future directions," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4316–4336, Jul. 2021.
- [5] J. Polanski, "Unsupervised learning in drug design from self-organization to deep chemistry," *Int. J. Mol. Sci.*, vol. 23, no. 5, 2022, Art. no. 2797.
- [6] F.-Y. Sun, J. Hoffmann, V. Verma, and J. Tang, "Infograph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization," in *Proc. J. Int. Conf. Learn. Representations*, 2020, pp. 1–16.
- [7] Y. Yan, E. Ricci, R. Subramanian, G. Liu, O. Lanz, and N. Sebe, "A multi-task learning framework for head pose estimation under target motion," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 6, pp. 1070–1083, Jun. 2016.
- [8] R. Zhang, H. Zhang, and X. Li, "Robust multi-task learning with flexible manifold constraint," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 6, pp. 2150–2157, Jun. 2021.
- [9] S. Vandenhende, S. Georgoulis, W. Van Gansbeke, M. Proesmans, D. Dai, and L. Van Gool, "Multi-task learning for dense prediction tasks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3614–3633, Jul. 2022.
- [10] S. Guo, O. Zoeter, and C. Archambeau, "Sparse Bayesian multi-task learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2011, vol. 24, pp. 1755–1763.
- [11] Y. Zhang and Q. Yang, "Learning sparse task relations in multi-task learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, vol. 31, no. 1, pp. 2914–2920.
- [12] Y. Zhang and Q. Yang, "A survey on multi-task learning," *IEEE Trans. Knowl. data Eng.*, vol. 34, no. 12, pp. 5586–5609, Dec. 2022.
- [13] I. Misra, A. Shrivastava, A. Gupta, and M. Hebert, "Cross-stitch networks for multi-task learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 3994–4003.
- [14] Y. Lu, A. Kumar, S. Zhai, Y. Cheng, T. Javidi, and R. Feris, "Fully-adaptive feature sharing in multi-task networks with applications in person attribute classification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5334–5343.
- [15] M. Long, Z. Cao, J. Wang, and P. S. Yu, "Learning multiple tasks with multilinear relationship networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 1593–1602.
- [16] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn, "Gradient surgery for multi-task learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 5824–5836.
- [17] C. Fifty, E. Amid, Z. Zhao, T. Yu, R. Anil, and C. Finn, "Efficiently identifying task groupings for multi-task learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 27503–27516.
- [18] N. Gönitz, C. Widmer, G. Zeller, A. Kahles, G. Rätsch, and S. Sonnenburg, "Hierarchical multitask structured output learning for large-scale sequence segmentation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2011, pp. 2690–2698.
- [19] L. Liu, T. Zhou, G. Long, J. Jiang, and C. Zhang, "Learning to propagate for graph meta-learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 1039–1050.
- [20] S. Liu, M. Qu, Z. Zhang, H. Cai, and J. Tang, "Structured multi-task learning for molecular property prediction," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2022, pp. 8906–8920.
- [21] D. Belanger and A. McCallum, "Structured prediction energy networks," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 983–992.
- [22] A. Kumar and H. Daume III, "Learning task grouping and overlap in multi-task learning," in *Proc. Int. Conf. Mach. Learn.*, 2012, pp. 1723–1730.
- [23] T. Evgeniou, C. A. Micchelli, M. Pontil, and J. Shawe-Taylor, "Learning multiple tasks with Kernel methods," *J. Mach. Learn. Res.*, vol. 6, pp. 615–637, 2005.
- [24] X. Niu, Y. Sun, and J. Sun, "Latent group structured multi-task learning," in *Proc. 52nd Asilomar Conf. Signals, Syst., Comput.*, 2018, pp. 850–854.
- [25] D. Szklarczyk et al., "STRING v11: Protein–protein association networks with increased coverage, supporting functional discovery in genome-wide experimental datasets," *Nucleic acids Res.*, vol. 47, no. D1, pp. D607–D613, 2019.
- [26] C. Glymour, K. Zhang, and P. Spirtes, "Review of causal discovery methods based on graphical models," *Front. Genet.*, vol. 10, 2019, Art. no. 524.

- [27] E. V. Bonilla, K. Chai, and C. Williams, "Multi-task Gaussian process prediction," in *Proc. Adv. Neural Inf. Process. Syst.*, 2007, vol. 20, pp. 153–160.
- [28] Y. Zhang and D.-Y. Yeung, "A regularization approach to learning task relationships in multitask learning," *ACM Trans. Knowl. Discov. From Data*, vol. 8, pp. 1–31, 2014.
- [29] J. Ma, Z. Zhao, X. Yi, J. Chen, L. Hong, and E. H. Chi, "Modeling task relationships in multi-task learning with multi-gate mixture-of-experts," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 1930–1939.
- [30] F. Zhuang et al., "A comprehensive survey on transfer learning," *Proc. IEEE*, vol. 109, no. 1, pp. 43–76, Jan. 2021.
- [31] T. Kato, H. Kashima, M. Sugiyama, and K. Asai, "Multi-task learning via conic programming," in *Proc. Adv. Neural Inf. Process. Syst.*, 2007, pp. 737–744.
- [32] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [33] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [34] J. Zhou et al., "Graph neural networks: A review of methods and applications," *AI open*, vol. 1, pp. 57–81, 2020.
- [35] S. Wu, F. Sun, W. Zhang, X. Xie, and B. Cui, "Graph neural networks in recommender systems: A survey," *ACM Comput. Surv.*, vol. 55, no. 5, pp. 1–37, 2022.
- [36] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, "Spectral networks and locally connected networks on graphs," in *Proc. J. Int. Conf. Learn. Representations*, 2014, pp. 1–14.
- [37] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, vol. 29, pp. 3844–3852.
- [38] M. Welling and T. N. Kipf, "Semi-supervised classification with graph convolutional networks," in *Proc. J. Int. Conf. Learn. Representations*, 2016, pp. 1–14.
- [39] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. J. Int. Conf. Learn. Representations*, 2018, pp. 1–12.
- [40] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 1025–1035.
- [41] Z. Wu et al., "MoleculeNet: A benchmark for molecular machine learning," *Chem. Sci.*, vol. 9, no. 2, pp. 513–530, 2018.
- [42] O. Wieder et al., "A compact review of molecular property prediction with graph neural networks," *Drug Discov. Today: Technol.*, vol. 37, pp. 1–12, 2020.
- [43] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proc. ACM Knowl. Discov. Data Mining*, 2018, pp. 974–983.
- [44] W. Fan et al., "Graph neural networks for social recommendation," in *Proc. World Wide Web Conf.*, 2019, pp. 417–426.
- [45] J. Qiu, J. Tang, H. Ma, Y. Dong, K. Wang, and J. Tang, "DeepInf: Social influence prediction with deep learning," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2110–2119.
- [46] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, vol. 31, pp. 5171–5181.
- [47] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1263–1272.
- [48] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proc. J. Int. Conf. Learn. Representations*, 2019, pp. 1–17.
- [49] N. T. Huang and S. Villar, "A short tutorial on the Weisfeiler-Lehman test and its variants," in *Proc. ICASSP 2021-2021 IEEE Int. Conf. Acoust., Speech Signal Process.*, 2021, pp. 8533–8537.
- [50] M. Qu, Y. Bengio, and J. Tang, "GMNN: Graph Markov neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 5241–5250.
- [51] C. Sutton et al., "An introduction to conditional random fields," *Foundations Trends Mach. Learn.*, vol. 4, no. 4, pp. 267–373, 2012.
- [52] R. M. Neal and G. E. Hinton, "A view of the em algorithm that justifies incremental, sparse, and other variants," in *Learning in Graphical Models*. Berlin, Germany: Springer, 1998, pp. 355–368.
- [53] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *Proc. J. Int. Conf. Learn. Representations*, 2014, pp. 1–14.
- [54] S. Gershman and N. Goodman, "Amortized inference in probabilistic reasoning," in *Proc. Annu. Meeting Cogn. Sci. Soc.*, 2014, vol. 36, no. 36, pp. 517–522.
- [55] M. Qu, H. Cai, and J. Tang, "Neural structured prediction for inductive node classification," in *Proc. J. Int. Conf. Learn. Representations*, 2022, pp. 1–21.
- [56] T. D. Le, T. Hoang, J. Li, L. Liu, H. Liu, and S. Hu, "A fast PC algorithm for high dimensional causal discovery with multi-core PCs," *IEEE/ACM Trans. Comput. Biol. Bioinf.*, vol. 16, no. 5, pp. 1483–1495, Sep./Oct. 2019.
- [57] P. Spirtes and K. Zhang, "Causal discovery and inference: Concepts and recent methodological advances," in *Applied Informatics*, Berlin, Germany: SpringerOpen, vol. 3, no. 1, 2016, Art. no. 3.
- [58] Y. He et al., "CausPref: Causal preference learning for out-of-distribution recommendation," in *Proc. ACM Web Conf. 2022*, pp. 410–421.
- [59] A. Hägele, J. Rothfuss, L. Lorch, V. R. Somnath, B. Schölkopf, and A. Krause, "BaCaDI: Bayesian causal discovery with unknown interventions," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2023, pp. 1411–1436.
- [60] J. Pearl, "Causal inference in statistics: An overview," *Statist. Surv.*, vol. 3, pp. 96–146, 2009.
- [61] X. Zheng, B. Aragam, P. K. Ravikumar, and E. P. Xing, "DAGs with NO TEARS: Continuous optimization for structure learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, vol. 31, pp. 9492–9503.
- [62] S. Zhu, I. Ng, and Z. Chen, "Causal discovery with reinforcement learning," in *Proc. J. Int. Conf. Learn. Representations*, 2020, pp. 1–17.
- [63] T. Kyono, Y. Zhang, and M. van der Schaar, "CASTLE: Regularization via auxiliary causal graph discovery," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 1501–1512.
- [64] C. Cundy, A. Grover, and S. Ermon, "BCD Nets: Scalable variational approaches for Bayesian causal discovery," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 7095–7110.
- [65] I. Ng, A. Ghassami, and K. Zhang, "On the role of sparsity and DAG constraints for learning linear DAGs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 17943–17954.
- [66] B. Huang et al., "Causal discovery from heterogeneous/nonstationary data," *J. Mach. Learn. Res.*, vol. 21, no. 89, pp. 1–53, 2020.
- [67] S. Liu, "Exploration on deep drug discovery: Representation and learning," *Tech. Rep.*, TR1854, 2018.
- [68] Y. LeCun, S. Chopra, R. Hadsell, M. Ranzato, and F. Huang, "A tutorial on energy-based learning," *Predicting Structured Data*, vol. 1, 2006, pp. 1–59.
- [69] I. Tschantzaris, T. Joachims, T. Hofmann, Y. Altun, and Y. Singer, "Large margin methods for structured and interdependent output variables," *J. Mach. Learn. Res.*, vol. 6, no. 9, pp. 1453–1484, 2005.
- [70] S. Sarawagi and R. Gupta, "Accurate max-margin training for structured output spaces," in *Proc. 25th Int. Conf. Mach. Learn.*, 2008, pp. 888–895.
- [71] D. Kocov, C. Vens, J. Struyf, and S. Džeroski, "Tree ensembles for predicting structured outputs," *Pattern Recognit.*, vol. 46, no. 3, pp. 817–833, 2013.
- [72] M. Arbel, L. Zhou, and A. Gretton, "Generalized energy based models," in *Proc. J. Int. Conf. Learn. Representations*, 2021, pp. 1–36.
- [73] M. Glymour, J. Pearl, and N. P. Jewell, *Causal Inference in Statistics: A Primer*. Hoboken, NJ, USA: Wiley, Inc., 2016.
- [74] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUS)," in *Proc. J. Int. Conf. Learn. Representations*, 2016, pp. 1–14.
- [75] E. Nijkamp et al., "Learning energy-based model with flow-based backbone by neural transport MCMC," 2020, *arXiv:2006.06897*.
- [76] M. Gutmann and A. Hyvärinen, "Noise-contrastive estimation: A new estimation principle for unnormalized statistical models," in *Proc. 13th Int. Conf. Artif. Intell. Statist., Workshop Conf. Proc.*, 2010, pp. 297–304.
- [77] M. Yang, F. Liu, Z. Chen, X. Shen, J. Hao, and J. Wang, "CausalVAE: Disentangled representation learning via neural structural causal models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 9593–9602.
- [78] D. P. Kingma, "Adam: A method for stochastic optimization," in *Proc. J. Int. Conf. Learn. Representations*, 2015, pp. 1–15.
- [79] S. Geman and D. Geman, "Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-6, no. 6, pp. 721–741, Nov. 1984.
- [80] Y. Yu, J. Chen, T. Gao, and M. Yu, "DAG-GNN: DAG structure learning with graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7154–7163.

- [81] A. Jaber, M. Kocaoglu, K. Shanmugam, and E. Bareinboim, "Causal discovery from soft interventions with unknown targets: Characterization and learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 9551–9561.
- [82] C. Squires, Y. Wang, and C. Uhler, "Permutation-based causal structure learning with unknown intervention targets," in *Proc. Conf. Uncertainty Artif. Intell.*, 2020, pp. 1039–1048.
- [83] P. W. Holland, K. B. Laskey, and S. Leinhardt, "Stochastic blockmodels: First steps," *Social Netw.*, vol. 5, no. 2, pp. 109–137, 1983.
- [84] T. Kamishima, "Nantonac collaborative filtering: Recommendation based on order responses," in *Proc. 9th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2003, pp. 583–588.
- [85] B. Bakker and T. M. Heskes, "Task clustering and gating for Bayesian multitask learning," *J. Mach. Learn. Res.*, vol. 4, pp. 83–99, 2003.
- [86] D. Mendez et al., "ChEMBL: Towards direct deposition of bioassay data," *Nucleic Acids Res.*, vol. 47, no. D1, pp. D930–D940, 2019.
- [87] J.-A. Désidéri, "Multiple-gradient descent algorithm (MGDA) for multi-objective optimization," *Comptes Rendus Mathématique*, vol. 350, no. 5–6, pp. 313–318, 2012.
- [88] A. Kendall, Y. Gal, and R. Cipolla, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7482–7491.
- [89] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich, "GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 794–803.
- [90] S. Liu, E. Johns, and A. J. Davison, "End-to-end multi-task learning with attention," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 1871–1880.
- [91] S. Liu, Y. Liang, and A. Gitter, "Loss-balanced task weighting to reduce negative transfer in multi-task learning," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 01, pp. 9977–9978.
- [92] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 3538–3545.
- [93] U. Alon and E. Yahav, "On the bottleneck of graph neural networks and its practical implications," in *Proc. J. Int. Conf. Learn. Representations*, 2021, pp. 1–16.



best research papers WWW'21 and best-ranked papers ICDM'21.

Xixun Lin received the PhD degree from the Institute of Information Engineering (IIE), Chinese Academy of Sciences, in 2022. He is currently an assistant professor with IIE. He has authored or coauthored several papers in *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Neural Networks and Learning Systems*, *ACM Transactions on Information Systems*, *Public Relations*, *ICML*, *WWW*, *AAAI* and *SIGIR*. His research interests include graph representation learning and trustworthy machine learning. He has been awarded candidates of



Qing Yu received the BS degree from the China University of Mining and Technology in 2023. She is currently working toward master's degree with Wuhan University. She is also a research intern with Baidu Inc. Her research interests focus on graph neural network, causal discovery, recommendation system and information retrieval.



Yanan Cao received the PhD degree from the Institute of Computing Technology, Chinese Academy of Sciences, in 2012. She is currently a professor with Institute of Information Engineering, Chinese Academy of Sciences. She has authored or coauthored more than 70 papers, including *AAAI*, *NeurIPS*, *IJCAI*, *ACL*, *WWW*. Her research interests include social network analysis and natural language processing. She won the best paper award of PAKDD-20.



KDD Cup 2019. His research interests includes information retrieval, recommender systems, and reinforcement learning.

Lixin Zou received the PhD in computer science & technology from Tsinghua University, in 2020. He is currently an associate professor with the School of Cyber Science and Engineering, Wuhan University. Prior to his current position, he was a staff algorithm engineer with Baidu Search Science team from Jul. 2020 to Nov. 2022. He has authored or coauthored more than 20 research papers in top-tier academic conferences and journals, including *SIGIR*, *KDD*, *WWW*, *NeurIPS*, *WSDM*, *IJCAI*, *TWeb*, and *CIKM*. He had led a team and won the runner up award of



analysis and graph mining. He won the outstanding doctoral dissertation of Chinese Academy of Sciences in 2014, the best paper award of ICCS-14, and the best student paper awards of IJCNN-17 and ICDM-21.

Chuan Zhou (Member, IEEE) received the PhD degree from the Chinese Academy of Sciences in 2013. He is currently an associate professor with the Academy of Mathematics and Systems Science, Chinese Academy of Sciences, and the University of Chinese Academy of Sciences. He has authored or coauthored more than 100 papers, including *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Neural Networks and Learning Systems*, *ICDM*, *AAAI*, *NeurIPS*, *CIKM*, *IJCAI* and *WWW*. His research interests include social network



actions on *Neural Networks and Learning Systems*, *IEEE Transactions on Multimedia*, *ACM Transactions on Knowledge Discovery from Data*, *NeurIPS*, *WWW*, and *KDD*. His current research interests include data mining and machine learning. He was the recipient of SDM'18 Best Paper Award in Data Science Track, IJCNN'17 Best Student Paper Award, and ICDM'14 Best Paper Candidate Award. He is also an associate editor of the *ACM Transactions on Knowledge Discovery from Data (TKDD)* and *Neural Networks (NN)*.

Jia Wu (Senior Member, IEEE) received the PhD degree in computer science from the University of Technology Sydney, Australia. He is currently the research director for the AI-enabled Processes (AIP) Research Centre and an ARC DECRA fellow with the School of Computing, Macquarie University, Sydney, Australia. Since 2009, he has authored or coauthored more than 100 refereed journal and conference papers, including *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Neural Networks and Learning Systems*, *IEEE Transactions on*



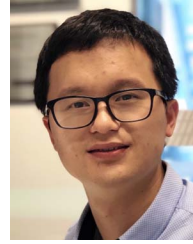
actions on *Information Systems*, *ACM Transactions on Asian and Low-Resource Language Information Processing*, *Information Processing & Management* and *Journal of the Association for Information Science and Technology*. His research won the *SIGIR* 2016 Best Student Paper Honorable Mention and *TKDE* Featured Spotlight Paper.

Chenliang Li is currently a professor with the School of Cyber Science and Engineering, Wuhan University. He has authored or coauthored more than 90 research papers on leading academic conferences and journals such as *SIGIR*, *ACL*, *WWW*, *IJCAI*, *AAAI*, *IEEE Transactions on Knowledge and Data Engineering* and *ACM Transactions on Information Systems*. His research interests include information retrieval, recommender systems, natural language processing and social media analysis. He was an associate editor / Editorial Board Member for *ACM Transactions on*



Peng Zhang is currently a professor with Guangzhou University, China. He was an associate professor with the Chinese Academy of Sciences, a lecturer with the University of Technology Sydney, and a staff algorithm engineer with the Alibaba Group and Ant Group. He has been researching into data mining, data streams, and social network analysis, with more than 150 publications in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Neural Networks and Learning*

Systems, KDD, SIGIR, WWW, ICDM, AAAI, IJCAI, etc. He was on many program committees of international conferences, including (senior) PC member for KDD, ICLR, ICML, NeurIPS, IJCAI, and AAAI conferences. He was also the editorial board of *Springer Journal of Big Data*, and *Springer Annals of Data Science*.



Shirui Pan (Senior Member, IEEE) received the PhD degree in computer science from the University of Technology Sydney (UTS), Ultimo, NSW, Australia. He is currently a professor and an ARC Future fellow with the School of Information and Communication Technology, Griffith University, Australia. He has authored or coauthored extensively in top-tier journals and conferences, including *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Neural Networks and Learning*

Systems, NeurIPS, ICML, and KDD. His research interests include data mining and machine learning.